Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра **гуманитарных, математических и естественно научных дисциплин**

ПРАКТИЧЕСКАЯ РАБОТА

Дисциплина(модуль)/предмет **МДК.02.02**

Тема: **Проектирование стратегии ветвления Git Flow**

ВАРИАНТ № **3**

Студент (ка)/учащийся: **Браунков Матвей Константинович** / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: **Чудинова А.А.** _____ /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «____» _____ 202__г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ОПИСАНИЕ ЗАДАНИЯ.....4
2 ПОШАГОВЫЙ СЦЕНАРИЙ С КОМАНДАМИ GIT.....5
3 СХЕМА ВЕТВЛЕНИЯ.....7
4 ВОПРОС ПО БЕЗОПАСНОСТИ: ГДЕ ПОДКЛЮЧАТЬ SAST.....8
ЗАКЛЮЧЕНИЕ.....9
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....10

ВВЕДЕНИЕ

Цель работы: научиться проектировать стратегию ветвления Git Flow [1] для распределённой команды разработки и описывать жизненный цикл изменений от функциональной ветки до релиза и срочного исправления, а также определить место статического анализа кода (SAST) в этом цикле.

Работа выполнена по варианту 3. Для проверки сценария он был реально выполнен в учебном репозитории командами Git, вывод консоли приведён на рисунках.

1 ОПИСАНИЕ ЗАДАНИЯ

Вариант 3: АИС «Интеграционный шлюз интернет-магазина». Система синхронизирует каталог, остатки на складе и платёжный шлюз, релизы выходят еженедельно небольшими частями (микрорелизы). Над системой параллельно работают три группы разработчиков: платёжный модуль, складской учёт и каталог.

Нужно описать работу групп в отдельных feature-ветках с последующим слиянием в develop и еженедельный цикл выпуска через ветку release/v1.x, а также ответить, на каком этапе Git Flow подключаются SAST и сканирование уязвимостей.

Таблица 1 – Ветки стратегии Git Flow для варианта 3

Ветка	**Откуда создаётся**	**Назначение**	**Куда сливается**
main	–	Только выпущенные версии, каждая помечена тегом vX.Y.Z	–
develop	main	Интеграция готовых функций следующего релиза	release/*
feature/*	develop	Одна функция одной группы (payment, warehouse, catalog)	develop
release/v1.x	develop	Подготовка недельного релиза: версия, тесты, исправление мелких ошибок	main и develop
hotfix/*	main	Срочное исправление ошибки в выпущенной версии	main и develop

2 ПОШАГОВЫЙ СЦЕНАРИЙ С КОМАНДАМИ GIT

2.1 Подготовка репозитория и feature-ветки

Репозиторий создаётся с веткой main, первый коммит помечается тегом v1.0.0. От main создаётся постоянная ветка develop. Каждая группа создаёт собственную ветку от develop командой git checkout -b feature/<имя> develop и делает в ней коммиты, не мешая остальным [2].

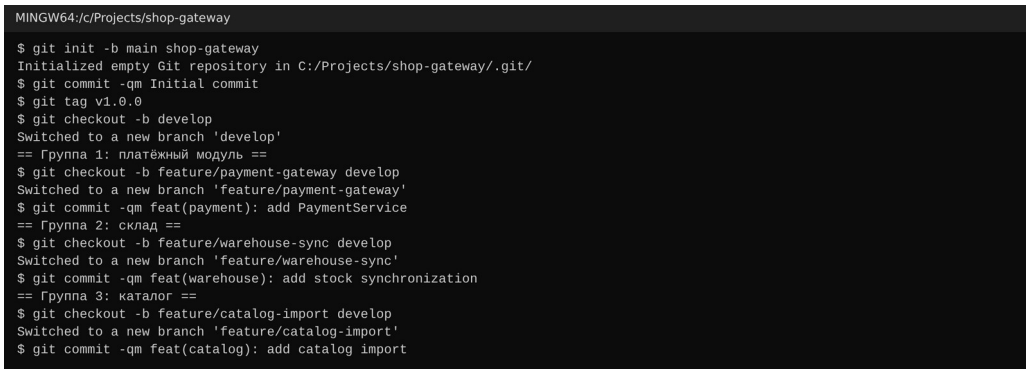


Рисунок 1 – Создание репозитория, ветки develop и трёх feature-веток

2.2 Слияние feature-веток в develop

Когда функция готова и прошла проверку через pull request, ветка вливается в develop с флагом --no-ff. Он создаёт отдельный коммит слияния, поэтому в истории видно, какая группа и что именно добавила.

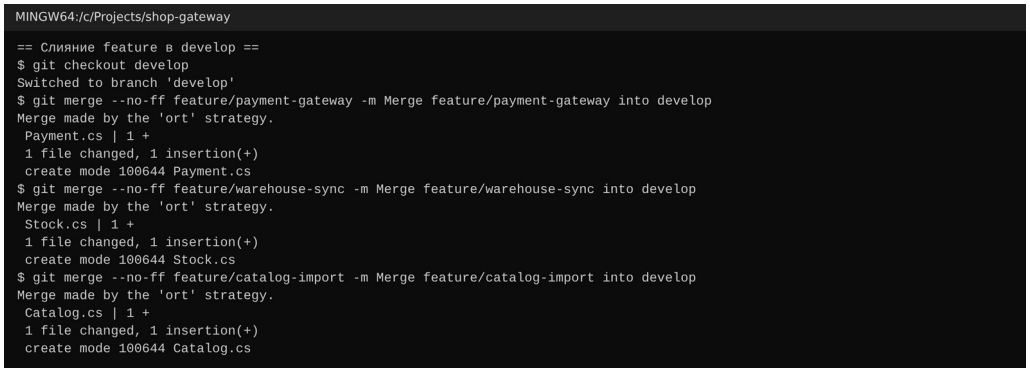


Рисунок 2 – Слияние feature-веток платежей, склада и каталога в develop

2.3 Еженедельный релиз release/v1.1

В день выпуска от develop создаётся ветка release/v1.1. В ней не добавляют новых функций, а только поднимают версию (SemVer [3]) и правят найденные ошибки. После проверки ветка вливается в main, на результат ставится тег v1.1.0, затем ветка вливается обратно в develop, чтобы правки релиза не потерялись, и удаляется.

```
MINGW64:/c/Projects/shop-gateway

== Релиз ==
$ git checkout -b release/v1.1 develop
Switched to a new branch 'release/v1.1'
$ git commit -qm chore(release): bump version to 1.1.0
$ git checkout main
Switched to branch 'main'
$ git merge --no-ff release/v1.1 -m Merge release/v1.1 into main
Merge made by the 'ort' strategy.
 Catalog.cs | 1 +
 Payment.cs | 1 +
 Stock.cs   | 1 +
 VERSION    | 1 +
 4 files changed, 4 insertions(+)
 create mode 100644 Catalog.cs
 create mode 100644 Payment.cs
 create mode 100644 Stock.cs
 create mode 100644 VERSION
$ git tag -a v1.1.0 -m Release 1.1.0
$ git checkout develop
Switched to branch 'develop'
$ git merge --no-ff release/v1.1 -m Merge release/v1.1 back into develop
Merge made by the 'ort' strategy.
 VERSION | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 VERSION
$ git branch -d release/v1.1
Deleted branch release/v1.1 (was 23966d8).
```

Рисунок 3 – Создание release/v1.1, слияние в main, тег v1.1.0, возврат в develop

2.4 Срочное исправление hotfix

Если в выпущенной версии найдена критичная ошибка (например, слишком короткий таймаут платёжного шлюза), от main создаётся hotfix/payment-timeout. После исправления ветка вливается в main с тегом v1.1.1 и в develop, чтобы ошибка не вернулась со следующим релизом [4].

```
MINGW64:/c/Projects/shop-gateway

== Hotfix ==
$ git checkout -b hotfix/payment-timeout main
Switched to a new branch 'hotfix/payment-timeout'
$ git commit -qm fix(payment): increase gateway timeout
$ git checkout main
Switched to branch 'main'
$ git merge --no-ff hotfix/payment-timeout -m Merge hotfix/payment-timeout into main
Merge made by the 'ort' strategy.
 Hotfix.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 Hotfix.txt
$ git tag -a v1.1.1 -m Hotfix 1.1.1
$ git checkout develop
Switched to branch 'develop'
$ git merge --no-ff hotfix/payment-timeout -m Merge hotfix/payment-timeout into develop
Merge made by the 'ort' strategy.
 Hotfix.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 Hotfix.txt
$ git branch -d hotfix/payment-timeout feature/payment-gateway feature/warehouse-sync feature/catalog-import
Deleted branch hotfix/payment-timeout (was a52ccd9).
Deleted branch feature/payment-gateway (was 40ef0ad).
Deleted branch feature/warehouse-sync (was 4551e7d).
Deleted branch feature/catalog-import (was ea6f74e).
```

Рисунок 4 – Hotfix: исправление, слияние в main, тег v1.1.1 и слияние в develop

3 СХЕМА ВЕТВЛЕНИЯ

На рисунке 5 показана схема потока изменений: слева направо идёт время, каждая строка – отдельная ветка, стрелки обозначают создание ветки и слияние.

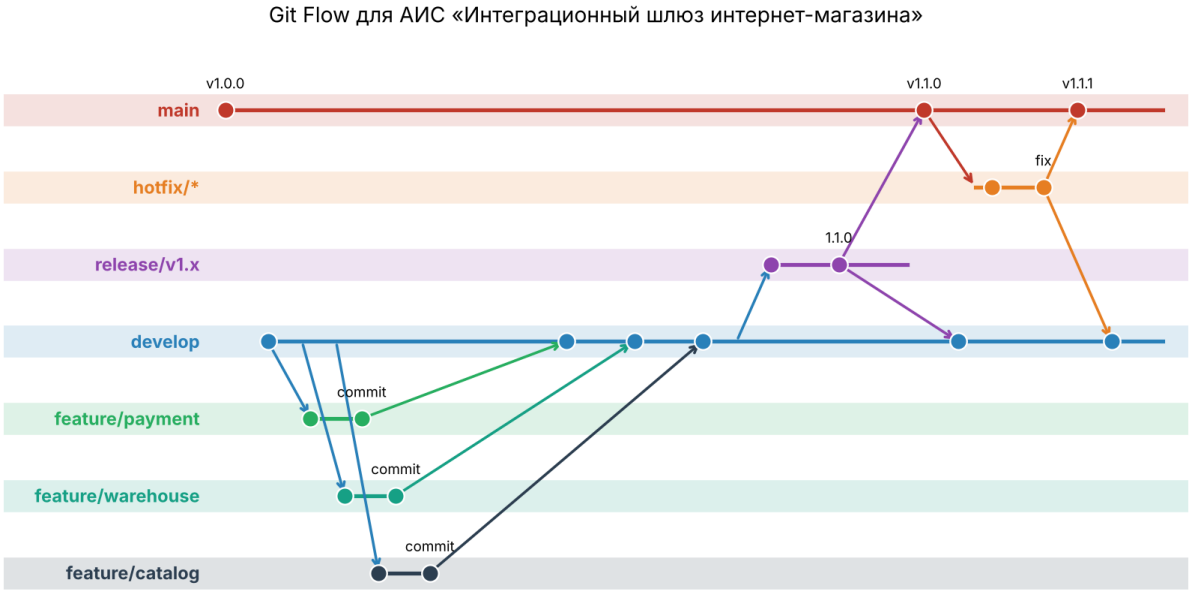


Рисунок 5 – Схема Git Flow для АИС «Интеграционный шлюз интернет-магазина»

Настоящий граф коммитов, полученный командой git log --graph после выполнения сценария, показан на рисунке 6.

```
MINGW64:/c/Projects/shop-gateway

$ git log --graph --oneline --decorate --all
*   174690d (HEAD -> develop) Merge hotfix/payment-timeout into develop
|\
* \   e495d6a Merge release/v1.1 back into develop
|\ \
| | | *   f68f2e1 (tag: v1.1.1, main) Merge hotfix/payment-timeout into main
| | | |\
| | | |/
| | |/|
| | * | a52ccd9 fix(payment): increase gateway timeout
| | |/
| | *   7c006a8 (tag: v1.1.0) Merge release/v1.1 into main
| | |\
| | |/
| |/|
| * | 23966d8 chore(release): bump version to 1.1.0
|/ /
* |   4681350 Merge feature/catalog-import into develop
|\ \
| * | ea6f74e feat(catalog): add catalog import
| |/
* |   457ec8f Merge feature/warehouse-sync into develop
|\ \
| * | 4551e7d feat(warehouse): add stock synchronization
| |/
* |   86fd4b7 Merge feature/payment-gateway into develop
|\ \
| |/
|/|
| * 40ef0ad feat(payment): add PaymentService
|/
* 9e001ae (tag: v1.0.0) Initial commit
$ git branch
* develop
  main
$ git tag
v1.0.0
v1.1.0
v1.1.1
```

Рисунок 6 – Граф коммитов и теги репозитория (git log --graph --oneline --decorate --all)

4 ВОПРОС ПО БЕЗОПАСНОСТИ: ГДЕ ПОДКЛЮЧАТЬ SAST

На каком этапе Git Flow подключаются SAST и сканирование уязвимостей и почему

SAST (статический анализ исходного кода [5]) нужно запускать как можно раньше, поэтому основной этап – pull request из feature-ветки в develop. Проверка запускается автоматически в CI, и слияние запрещено, пока нет «зелёного» результата (защищённые ветки [6]). Ошибка, найденная в одной функции одной группы, стоит дешевле, чем та же ошибка в готовом релизе. Для платёжного модуля это особенно важно: SQL-инъекции, секреты в коде и небезопасная работа с картами не должны попадать в общую ветку develop.

Второй запуск SAST выполняется на ветке release/v1.x перед слиянием в main: проверяется собранная версия целиком, включая взаимодействие модулей. Здесь же запускают сканирование зависимостей (SCA) на известные уязвимости и анализ образов, а при критичных находках релиз не выпускается. Для еженедельных микрорелизов такой «шлюз качества» должен быть автоматическим, иначе он замедлит выпуск [7].

Для ветки hotfix проверка сокращённая, но обязательная: быстрый SAST изменённых файлов, чтобы срочная правка не создала новую уязвимость. Кроме того, по расписанию (например, раз в ночь) полный анализ запускают на develop, так как новые правила и базы уязвимостей появляются и для старого кода. Примером инструмента является CodeQL [8].

Таблица 2 – Контрольные точки безопасности в цикле Git Flow

Этап	**Проверка**	**Условие продолжения**
PR feature → develop	SAST, поиск секретов, проверка стиля	Нет критичных и высоких находок
Ночная сборка develop	Полный SAST, SCA зависимостей	Новые находки заводятся как задачи
release/v1.x → main	SAST всей версии, SCA, сканирование образа	Релиз-блокирующих уязвимостей нет
hotfix → main	Быстрый SAST изменений	Нет новых находок в правке

ЗАКЛЮЧЕНИЕ

В работе спроектирована и проверена на практике стратегия Git Flow для АИС «Интеграционный шлюз интернет-магазина»: три группы работают в отдельных feature-ветках, слияния в develop идут через pull request, еженедельный релиз проходит через release/v1.x с тегом SemVer, а срочные ошибки исправляются через hotfix с обязательным возвратом в develop.

Определено, что SAST и сканирование уязвимостей нужно подключать на pull request в develop, на release-ветке перед выпуском и в сокращённом виде на hotfix. Так уязвимости находятся на раннем этапе, а релиз остаётся быстрым.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Atlassian. Gitflow Workflow [Электронный ресурс]. – URL: https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow (дата обращения: 30.10.2026).

2. Chacon S., Straub B. Pro Git [Электронный ресурс]. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 30.10.2026).

3. Semantic Versioning 2.0.0 [Электронный ресурс]. – URL: https://semver.org/lang/ru/ (дата обращения: 30.10.2026).

4. Driessen V. A successful Git branching model [Электронный ресурс]. – URL: https://nvie.com/posts/a-successful-git-branching-model/ (дата обращения: 30.10.2026).

5. OWASP. Source Code Analysis Tools [Электронный ресурс]. – URL: https://owasp.org/www-community/Source_Code_Analysis_Tools (дата обращения: 30.10.2026).

6. GitHub Docs. About protected branches [Электронный ресурс]. – URL: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches (дата обращения: 30.10.2026).

7. OWASP. DevSecOps Guideline [Электронный ресурс]. – URL: https://owasp.org/www-project-devsecops-guideline/ (дата обращения: 30.10.2026).

8. GitHub Docs. About code scanning with CodeQL [Электронный ресурс]. – URL: https://docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning-with-codeql (дата обращения: 30.10.2026).